Automated MoU Email Dispatch System for Windows: Using Installed Outlook

1. Overview

This Python script automates sending emails to investors or signatories based on the status of their Memorandums of Understanding (MoUs). It uses Microsoft Outlook installed on Windows machine to send emails and generates a detailed log of each automated run.

The script handles multiple email statuses, loads custom templates for each status, replaces placeholders in the templates with actual MoU details, optionally attaches files, and logs all actions in a structured Excel file (LogOfEmail.xlsx).

2. Functionality

The script performs the following steps:

1. **Load Excel**
   Reads the main Excel sheet (MailingAuto.xlsx) containing MoU details.
2. **Read Required Columns**
   - **Mandatory:** MoU No., Auth. Signatory (Email), Mail Status
   - **Optional:** IP Expert, IP Expert Mobile Number, Attachment
3. **Check Mail Status**
   The Mail Status column must **exactly match** one of the following (case-sensitive, no extra spaces):
   - Pending for Approvals
   - Project Commissioned
   - Construction Started
   - Incentives Related Issue
   - Land Related Issue
   - Land Required
   - Not Aware about the MoU
   - MoU On Hold
   - MoU Dropped
   - Other Issues
   - Policy Related Issues
   - Support Infrastructure Required

Each status is mapped to a corresponding template in the templates folder. Incorrect spelling or casing will cause the row to be skipped.

4. **Load Templates**
   Replaces placeholders in templates with actual values:
   - <MoU No> → MoU number from Excel
   - <IP Expert Name> → Name of the IP expert
   - <IP Expert Mobile No> → Mobile number of the IP expert
5. **Attach Files (Optional)**
   - The Attachment column in Excel lists filenames, provided by the user and separated by commas (if multiple)
   - All attachments must exist in the Attachments folder.
   - Missing files are skipped with a console warning; the script continues.
6. **Send Emails via Outlook**
   - Uses the account: risingrajasthan.mou1@rajasthan.gov.in.
   - Dry run mode simulates sending without actually sending emails.
   - If the Outlook account is not found, the console displays a warning.
7. **Log Results**
   - Each processed row is recorded in LogOfEmail.xlsx with:
     - MoU No.
     - Auth. Signatory (Email)
     - IP Expert Name
     - Status of mail (Sent, Dry run, Skipped, Failed)
     - Timestamp of run
   - Historical runs are preserved.

3. Folder and File Requirements

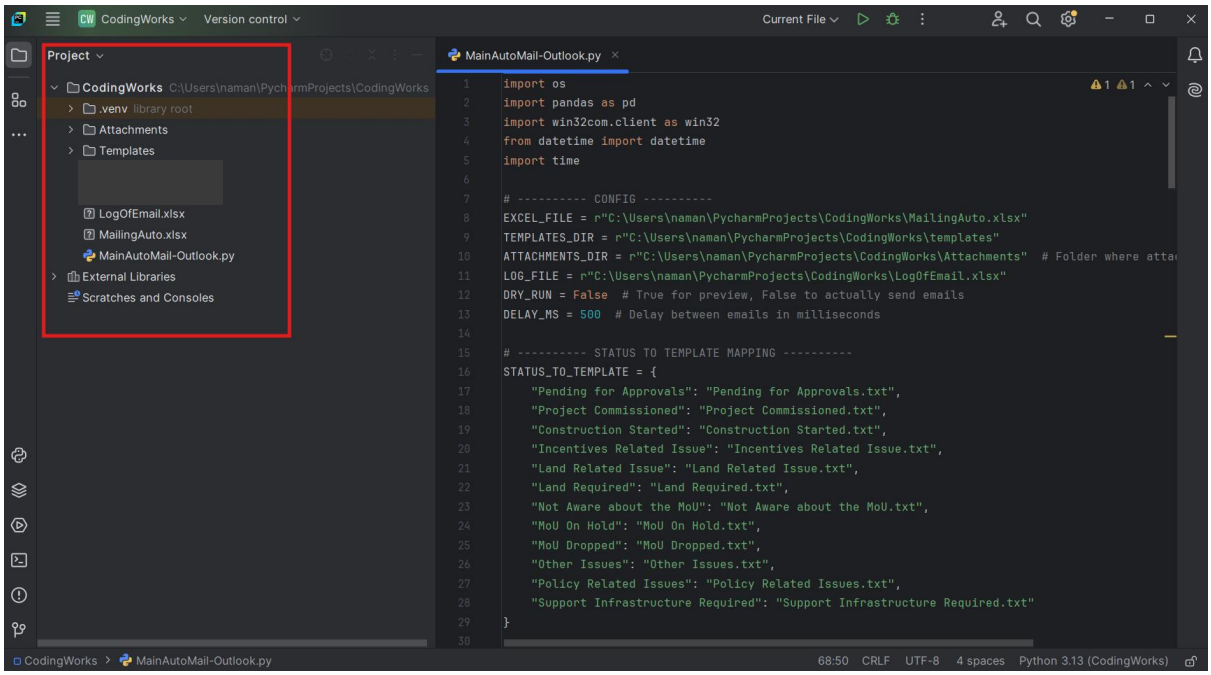
Required Folder structure for the code to function:

```
C:\Users\<username>\<PycharmInstallLocation> \<PycharmProjectName>
│
├─ AutoMail.py            # Main Python script
├─ MailingAuto.xlsx       # Excel sheet with MoU details
├─ templates              # Folder containing email templates
│   ├─ Pending for Approvals.txt
```

```
│   ├─ Project Commissioned.txt
│   ├─ Construction Started.txt
│   ├─ ...              # Other status templates
├─ Attachments          # Folder containing attachment files
│   ├─ file1.pdf
│   ├─ file2.xlsx
│   ├─ file3.docx
│   ├─ ...              # Other attachments
└─ LogOfEmail.xlsx      # Auto-generated email run log
```

Notes:

- All template files must be in .txt format.
- Mail Status values in Excel must exactly match the template keys.
- The Attachment column lists filenames, **relative to the Attachments folder**.
- Any missing files will be skipped with a console warning.



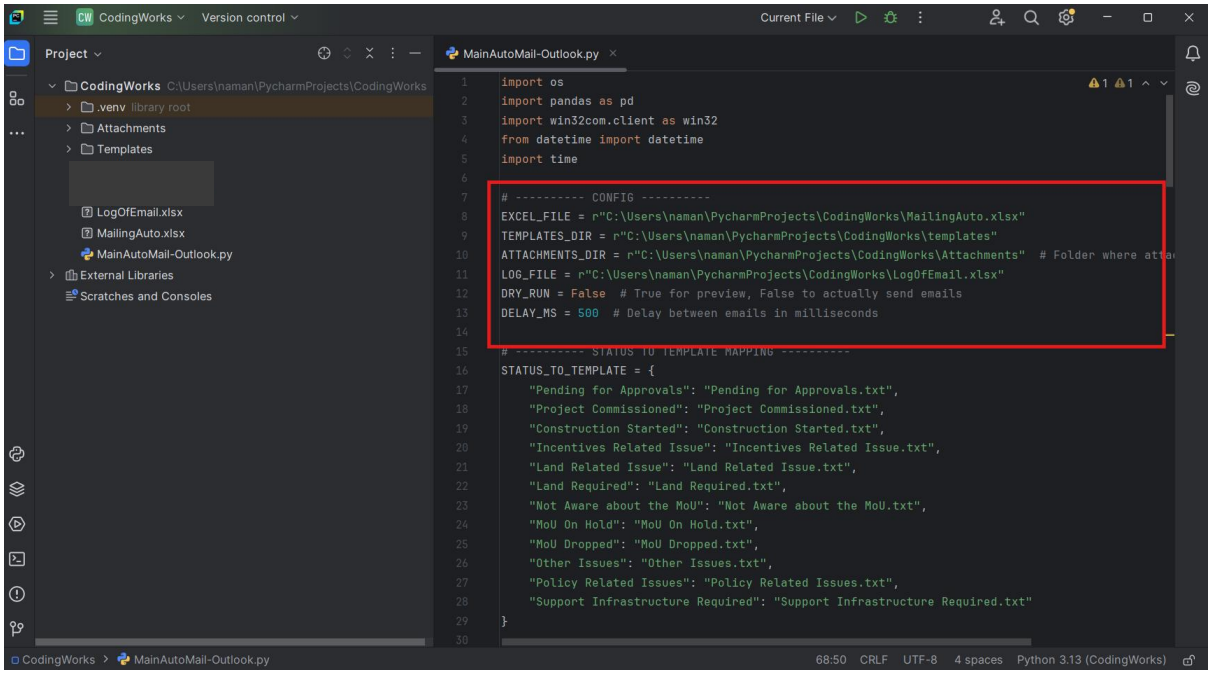
4. Environment Requirements

- Windows Operating System
- Python installed.
- PyCharm IDE or any Python editor (optional but recommended).

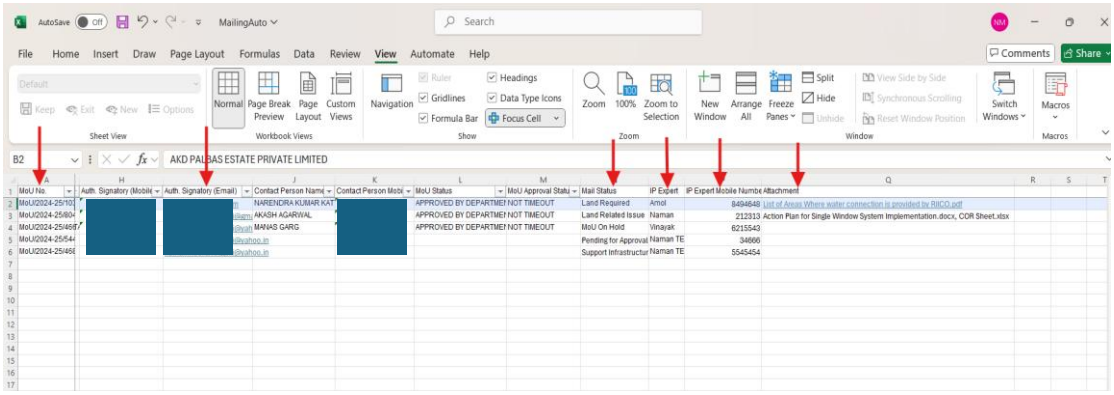
- Required Python packages: pandas, pywin32, openpyxl
- pip install pandas pywin32 openpyxl
- Microsoft Outlook installed and configured with the account risingrajasthan.mou1@rajasthan.gov.in.

5. How the Script Works

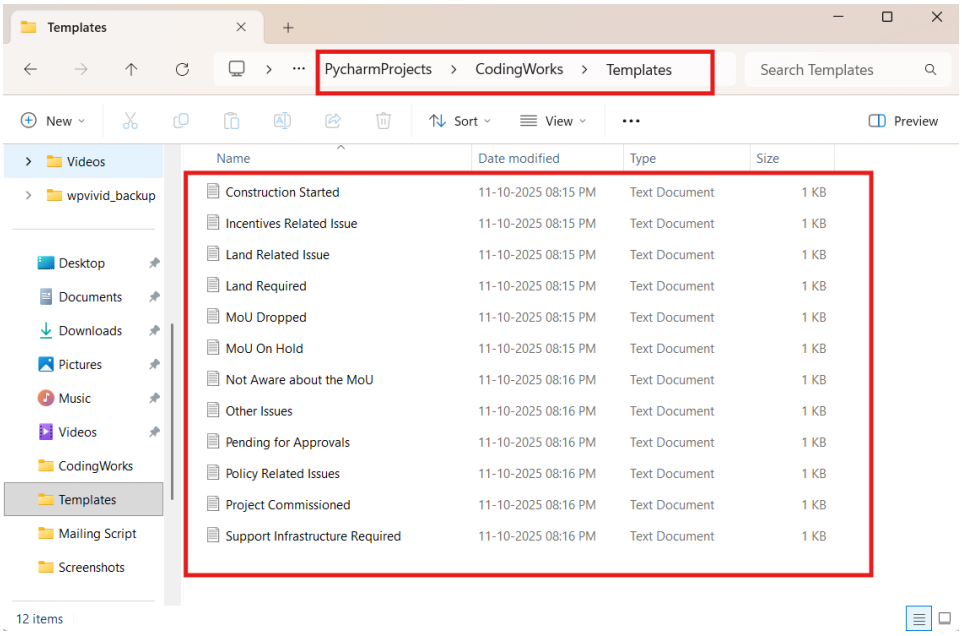
1. **Load Excel** – The script reads MailingAuto.xlsx into a panda DataFrame.



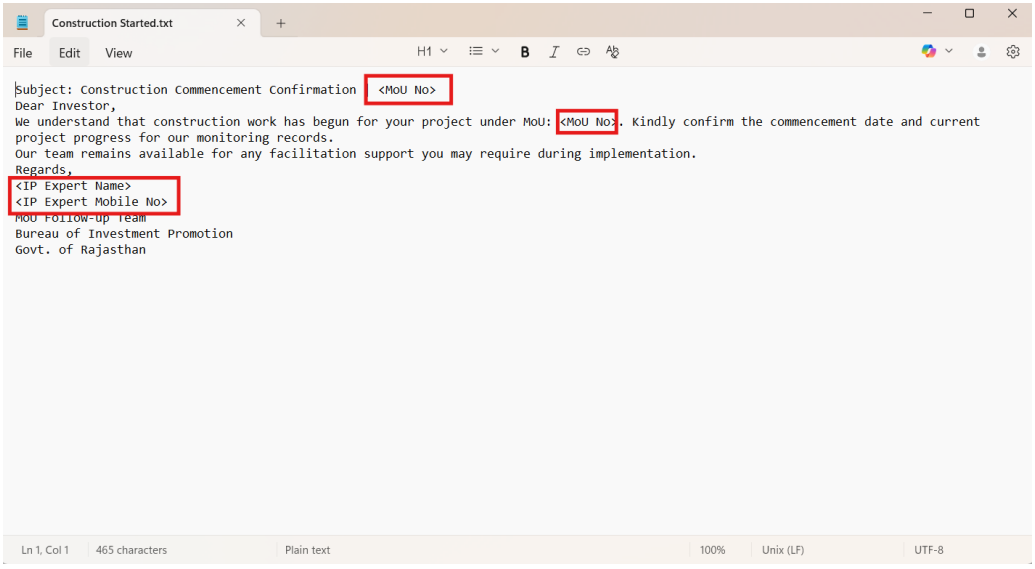
2. **Process Rows** – For each row:
   - Checks mandatory fields (MoU No., Auth. Signatory (Email), Mail Status)



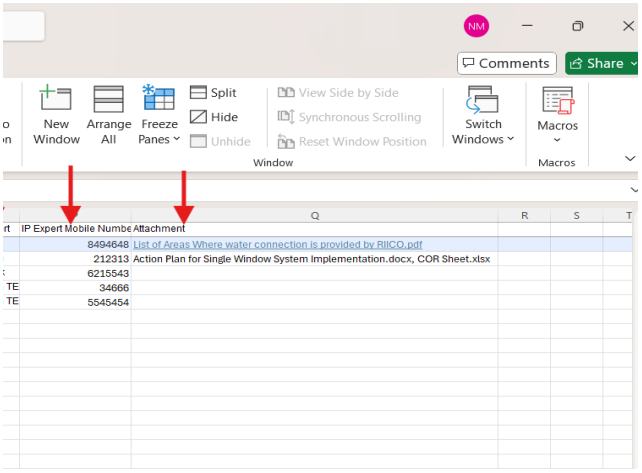
- Loads template based on Mail Status



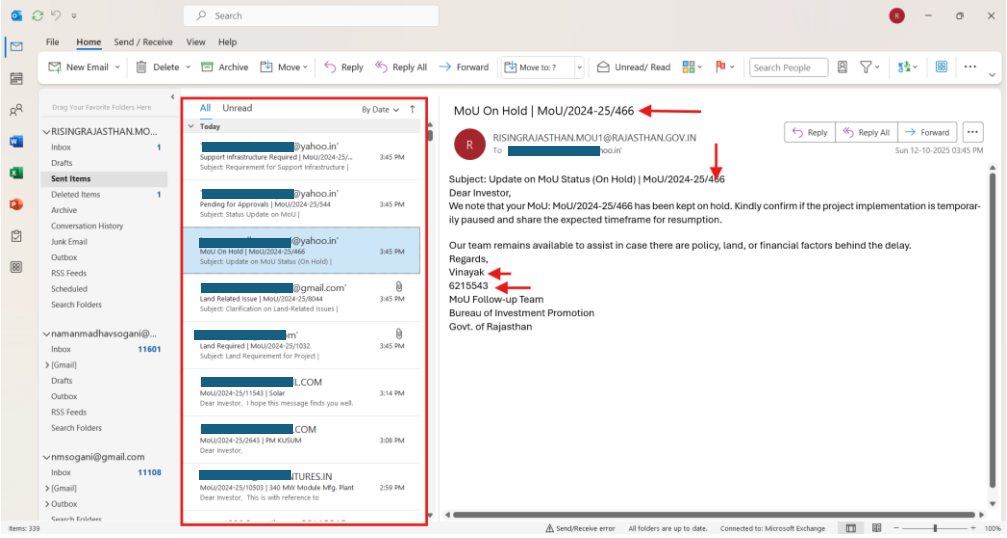
- Replaces placeholders with actual values



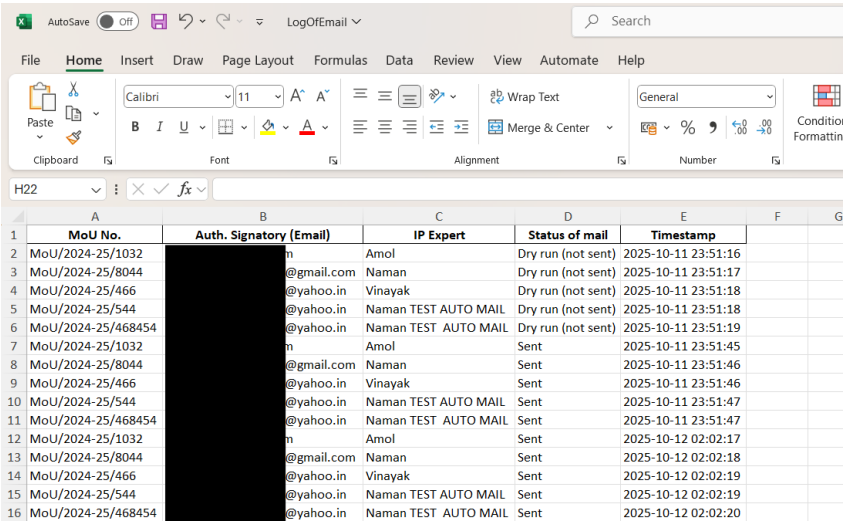
- Adds attachments if listed



3. **Send Email** – Using Outlook account:
    - Sends email (or previews in dry run mode)
    - If the account is missing, email may bounce back



4. **Log Results** – Each processed row is recorded with timestamp and status. Historical runs are preserved.



MoU No.	Auth. Signatory (Email)	IP Expert	Status of mail	Timestamp
MoU/2024-25/1032	[redacted]m	Amol	Dry run (not sent)	2025-10-11 23:51:16
MoU/2024-25/8044	[redacted]@gmail.com	Naman	Dry run (not sent)	2025-10-11 23:51:17
MoU/2024-25/466	[redacted]@yahoo.in	Vinayak	Dry run (not sent)	2025-10-11 23:51:18
MoU/2024-25/544	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Dry run (not sent)	2025-10-11 23:51:18
MoU/2024-25/468454	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Dry run (not sent)	2025-10-11 23:51:19
MoU/2024-25/1032	[redacted]m	Amol	Sent	2025-10-11 23:51:45
MoU/2024-25/8044	[redacted]@gmail.com	Naman	Sent	2025-10-11 23:51:46
MoU/2024-25/466	[redacted]@yahoo.in	Vinayak	Sent	2025-10-11 23:51:46
MoU/2024-25/544	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Sent	2025-10-11 23:51:47
MoU/2024-25/468454	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Sent	2025-10-11 23:51:47
MoU/2024-25/1032	[redacted]m	Amol	Sent	2025-10-12 02:02:17
MoU/2024-25/8044	[redacted]@gmail.com	Naman	Sent	2025-10-12 02:02:18
MoU/2024-25/466	[redacted]@yahoo.in	Vinayak	Sent	2025-10-12 02:02:19
MoU/2024-25/544	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Sent	2025-10-12 02:02:19
MoU/2024-25/468454	[redacted]@yahoo.in	Naman TEST AUTO MAIL	Sent	2025-10-12 02:02:20

6. Configuration Options

- **DRY_RUN**
    - True → Emails are previewed only, not sent
    - False → Emails are actually sent
- **DELAY_MS** – Delay between emails in milliseconds (default: 500ms)

7. Caveats and Limitations of Process:

1. **Windows Based** Operating system.

2. **Mail Status spelling:** Only rows with exact Mail Status values (case-sensitive) are processed. Typos will cause the row to be skipped.
3. **Outlook dependency:** Script requires Outlook installed and configured with the correct account.
4. **Email validation:** Recipient emails are **not verified**; invalid addresses may bounce back.
5. **Template accuracy:** Missing or misnamed template files cause skipped rows.
6. **Attachment location:** Files listed in Attachment column must exist in Attachments folder. Missing files are skipped.
7. **Mandatory columns:** Rows missing MoU No., Auth. Signatory (Email), or Mail Status are skipped.
8. **Concurrent edits:** Do not open MailingAuto.xlsx or LogOfEmail.xlsx while the script runs.
9. **Historical log:** Each run appends to the existing log with timestamp; do not manually delete rows unless necessary.

8. Steps for a New User

1. Install Python 3.x and required packages. PyCharm: The only Python IDE you need
2. Ensure Outlook is installed (Locally) and logged in with risingrajasthan.mou1@rajasthan.gov.in.
3. Place the script, Excel, templates, and attachments in the prescribed folder hierarchy (Section 3).
4. Open the script in PyCharm.
5. Configure DRY_RUN and DELAY_MS if needed.
6. Close Main excel file and the log file (if opened).
7. Run the script.
8. Check LogOfEmail.xlsx for results and any skipped rows.
9. Check in Outlook “Sent Folder” for the sent mails and “Inbox” for any bounce backs.

9. Summary

This automated mailing script allows government teams to efficiently communicate MoU-related updates to multiple stakeholders while maintaining a structured log for accountability. Proper setup, template management, and attachment handling are crucial for seamless operation. The results on a small batch of 5 mails were positive and the script performed as required.

Prepared By:
Naman Madhav Sogani
Intern, EY